

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 10

Searching

Storing and Retrieving

- So far, Linked Lists, LIFO, FIFO, Deque
- $O(1)$ insertion, deletion from either end
 - Or from anywhere, for linked lists
- For deque, $O(1)$ random access
- What is missing?
 - Search! Given a "search key" retrieve an element from the data structure with the same key (or report there is no such element)
- Search key often supports a total ordering ($=$, $<$, $>$, $\leq$, $\geq$ operators)
 - Numerical, lexicographical, ...
 - Full entry may be key + payload (e.g., key=roll no., payload=name)

Searching in a Vector

- Given a vector or array of (say) integers, check for the absence/presence of various numbers in the array (return an index, or -1)

```
int search(std::vector<float> A, float query);
```

- Simplest idea, in $\Theta(n)$:

```
for (int i=0; i<A.size(); i++)  
    if (A[i]==query) return i;  
return -1;
```

- Can we do better? Yes, *if the vector is sorted*, much better!

Binary Search

- Consider searching for a number, say 3.14 in an array of sorted numbers
- Let us look at the middle first!



- We can eliminate half of the array from consideration now!
- To search in the rest of the array, we *recurse*!
 - Look at the middle of the remaining array
 - Eliminate half again
- Until a match found (a middle element equals the target) or all of the array eliminated

Binary Search

```
int srch (const std::vector<float>& A, float query, int left, int right) {  
    if (right < left)  
        return -1;                // empty array: query not found  
    int mid = (left+right)/2;  
    if (query < A[mid])  
        return srch(A,query,left,mid-1);    // recurse to the left of mid  
    else if (A[mid] < query)  
        return srch(A,query,mid+1,right);    // recurse to the right of mid  
    else // query == A[mid]  
        return mid;                // found!  
}
```

Binary Search: Non-Recursive Version

- How did we use the implicit stack in the recursion?
- We didn't! Can rewrite as a loop with constant extra space!

```
int srch_loop(vector<float>& A, float query) {  
    int left = 0, right = A.size()-1, mid = (left+right)/2;  
    while (right >= left && query != A[mid]) {  
        if(query < A[mid]) right = mid-1;  
        else left = mid+1;  
        mid = (left+right)/2;  
    }  
    return (right < left) ? -1 : mid;  
}
```

- Aside: Such rewriting possible whenever it is a "tail recursion"

Tail Recursion

- In many cases of recursion, the return value is the same as what the recursive call returns
- Such a recursion can be rewritten as a loop
- Avoids overheads of function calls and returns
 - Like pushing/popping a frame onto/from the function-call stack, copying arguments and return values
- Compilers may often do such rewriting automatically

```
resultType f(inputType x) {  
  
    if (baseCase(x))  
        return handleBaseCase(x);  
  
    x = smallerProblem(x);  
  
    return f(x);  
}
```

```
resultType f(inputType x) {  
    while (!baseCase(x)) {  
        x = smallerProblem(x);  
    }  
    return handleBaseCase(x);  
}
```

Tail Recursion

```
!( right < left  
  || query == A[mid] )
```

```
int srch (std::vector<float>& A, float query, int left, int right) {  
    if (right < left)  
        return -1;  
    int mid = (left+right)/2;  
    if (query == A[mid])  
        return mid;  
  
    if (query < A[mid])  
        right = mid-1;  
    else  
        left = mid+1;  
  
    return srch(A,query,left,right);  
}
```

```
int srch_loop(vector<float>& A, float query) {  
    int left = 0, right = A.size()-1, mid;  
    while (right >= left  
           && query != A[mid=(left+right)/2]) {  
        if(query < A[mid]) right = mid-1;  
        else left = mid+1;  
    }  
    return (right < left) ? -1 : mid;  
}
```

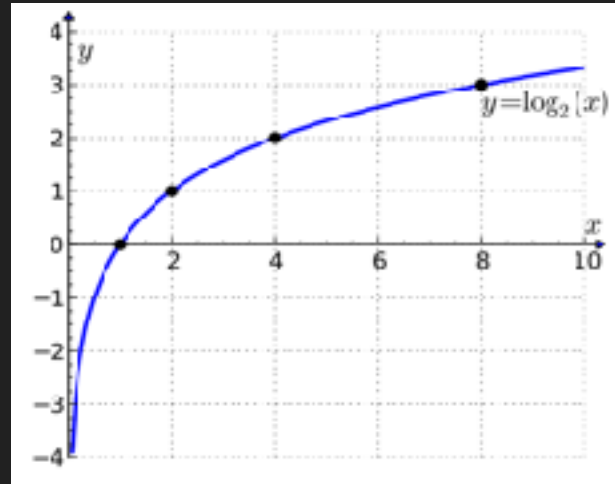
```
resultType f(inputType x) {  
    while (!baseCase(x)) {  
        x = smallerProblem(x);  
    }  
    return handleBaseCase(x);  
}
```


How Many Steps?

- How many comparisons in a binary search over n elements?
- Important point: in each level of recursion, n reduces by at least half
 - Verify! (Check even n , odd n , left half, right half.)
- Suppose $n \leq 2^k$
 - Input after t recursive calls is at most 2^{k-t} long. At most $k+1$ calls before empty input.
 - So $O(\log_2 n)$ recursive calls
- $\log_2 n = (\log_2 10) \log_{10} n = (3.32..) \log_{10} n$
 - For $O()$ notation, the base of the logarithm doesn't matter
- Binary search on a sorted array of n elements takes $O(\log n)$ steps

$O(\log n)$

- Log is a very slow growing function
- When n doubles, $\log_2 n$ increases by just 1
- When n is multiplied by a factor 1000, $\log_2 n$ increases by less than 10
 - $\log_2 (\text{million}) \leq 20$
 - $\log_2 (\text{billion}) \leq 30$
 - $\log_2 (\text{trillion}) \leq 40$
- The curve gets flatter (closer to horizontal) as n increases



Example: (Integer) Square Root

- Find $\lfloor \sqrt{x} \rfloor$ using binary search!
- Search in the implicit "sorted array" $\{0, 1, \dots, x\}$

```
int isqrt(int x, int left=0, int right=0) {
    if(right==0) right = x; // set range [0..x] if called without a range
    if (right < left)        // we just passed the square-root
        return right;       // right has the smaller number
    int mid = (left+right)/2;
    int sqr = mid*mid;       // to check  $\sqrt{x} \leq \text{mid}$ , will check  $x \leq \text{sqr}$ 
    if (x < sqr)
        return isqrt(x, left, mid-1);
    else if (sqr < x)
        return isqrt(x, mid+1, right);
    else // sqr == x
        return mid;
}
```

Example: Square Root

- Find $\sqrt{x}$ using binary search
- Search in the implicit "sorted array" of real numbers!

Adds one bit of precision in each iteration. Faster methods exist.

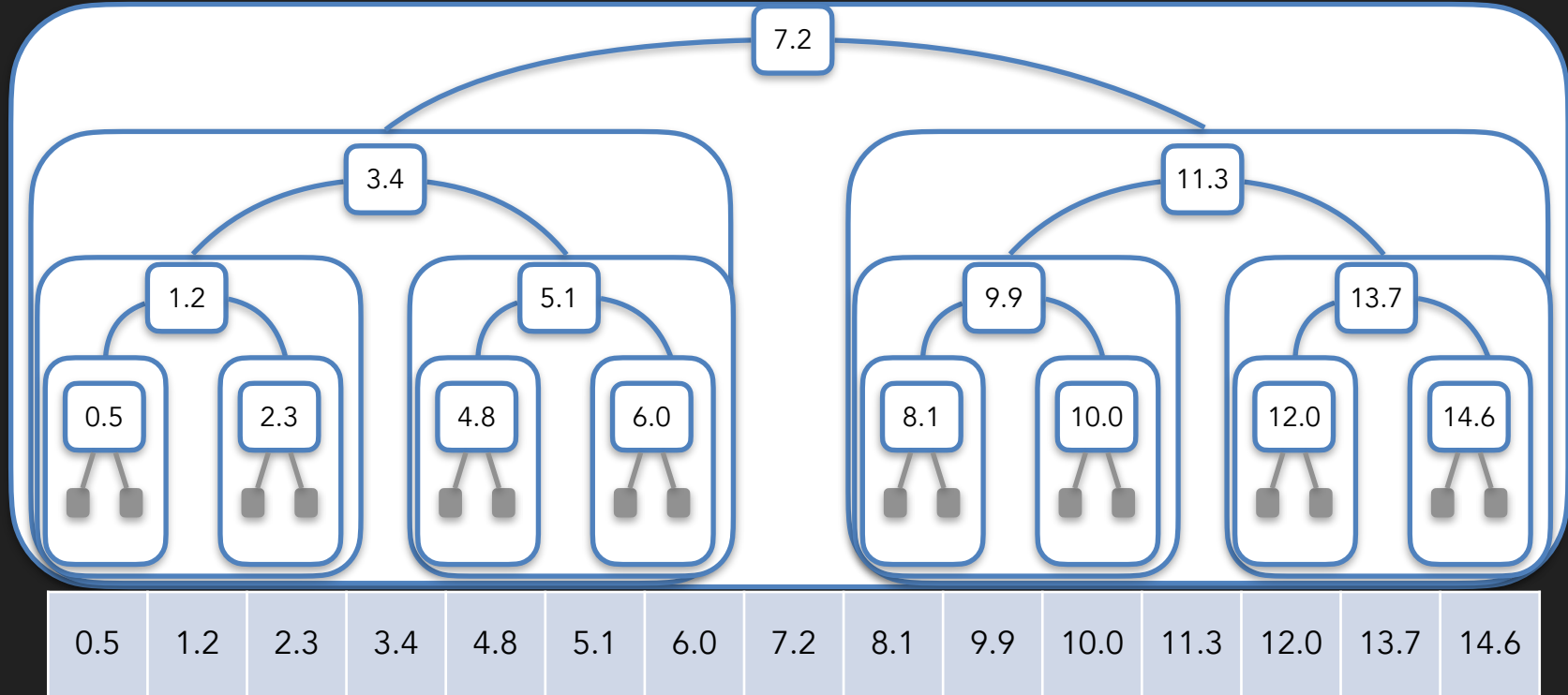
```
double sqrt(double x, double left=0, double right=0) { // assumes x >= 0
    if(right==0) right = std::max(1.0,x); //search [0,x], or if x<1, [0,1]
    double mid = (left+right)/2;
    if (mid==left || mid==right) //interval too small for double to bisect
        return mid;
    double sqr = mid*mid;
    if (x < sqr)
        return sqrt(x,left,mid);
    else if (x > sqr)
        return sqrt(x,mid,right);
    else // sqr == x
        return mid;
}
```

Binary Search Beyond Arrays

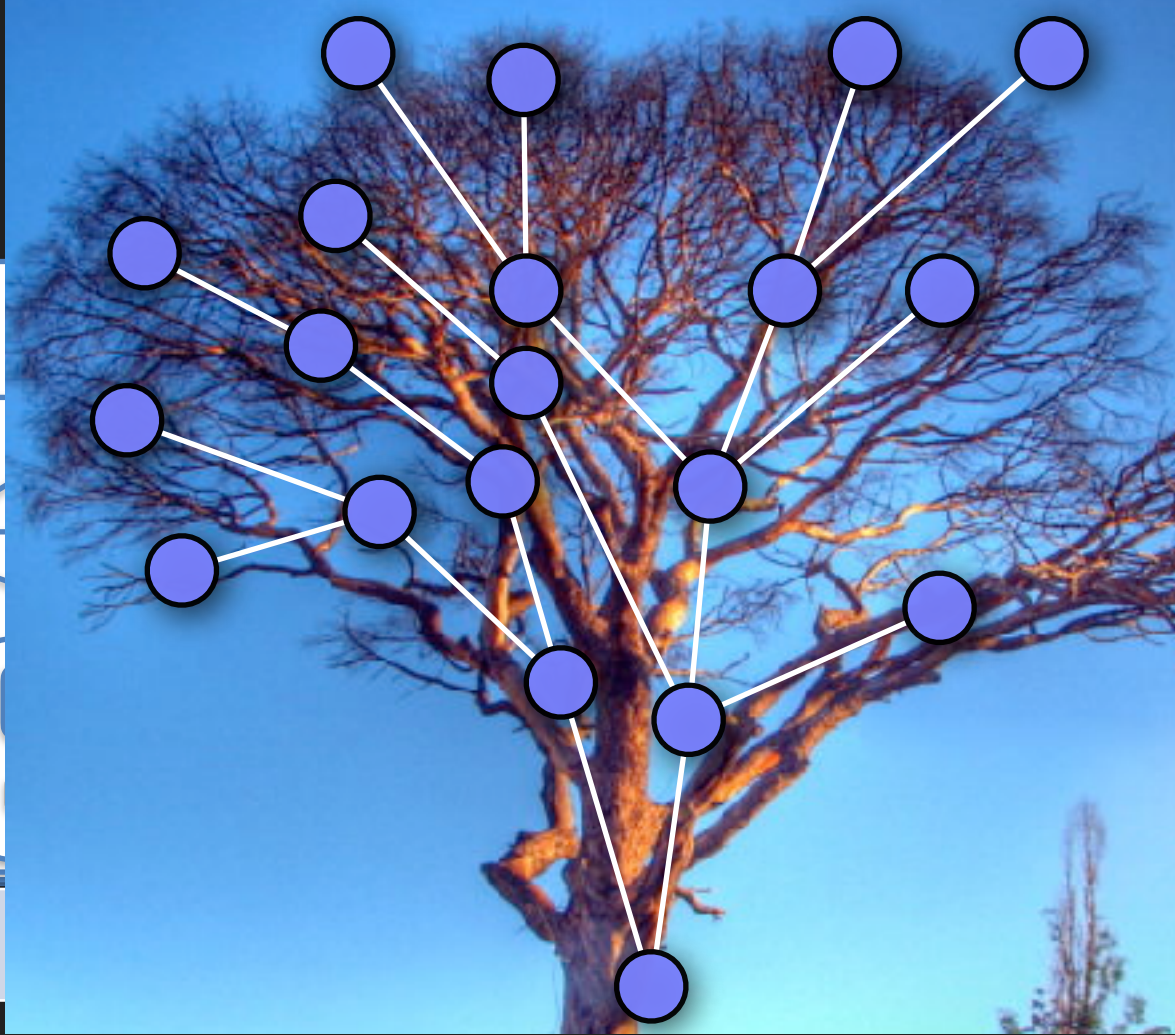
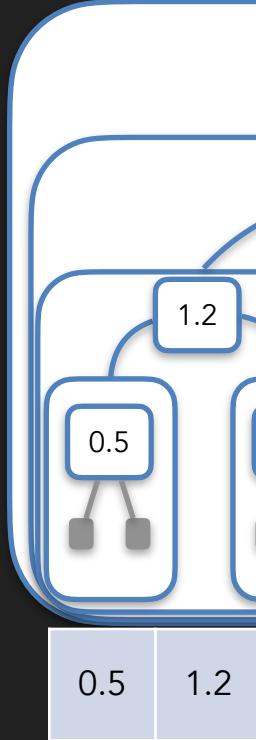
- Remember, an array or vector doesn't support quick insertions in the middle
 - Expensive ($O(n^2)$ time) to maintain a sorted array, if items keep arriving
- Can we keep data arranged so that we have the benefits of a sorted array plus quick insertions?
- That is,
 - $O(\log n)$ search
 - $O(n)$ time to output in sorted order
 - Plus, $O(\log n)$ insertion time (Later: can't hope for better)

Binary Search Beyond Arrays

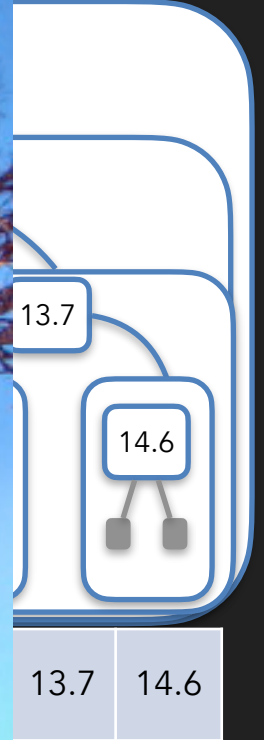
- Mimicking binary search: keep $A[\text{mid}]$ directly accessible. Keep the left and right halves in the same recursive structure



- Mimicking
and right h



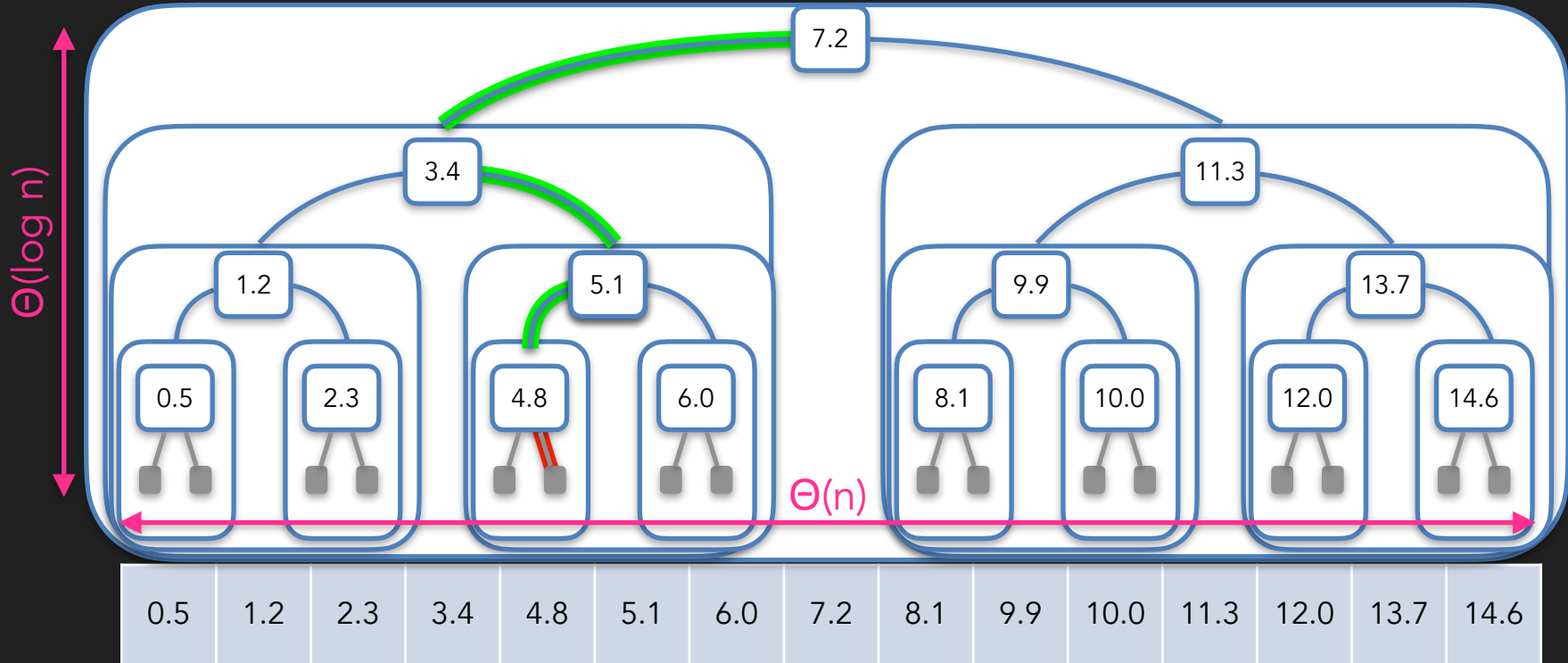
ep the left



Binary Search Trees

- Searching: Follow the binary-search path "down" the tree

query = 5.0



Binary Search Trees

- Coming up:
 - Anatomy of a Binary Tree
 - Mechanics of a Binary Search Tree:
 - Already saw: Searching in $O(\log n)$ time
 - Next: Inserting, deleting a few elements in $O(\log n)$ time
 - Later: Inserting, deleting any number of elements